ZYPHRON CLOUD

INFRASTRUCTURE & HOSTING SOLUTIONS

API DOCUMENTATION

Zyphron Free Dashboard - Complete Endpoint Reference

Document Ref:	**ZC-API-DOC-2026**
Base URL:	**http://108.181.193.242:8080/api**
Version:	**1.0**
Date:	**June 21, 2026**
Total Endpoints:	**130+**
Auth:	**JWT Bearer Token**

TABLE OF CONTENTS

1 Authentication & Overview

2 Auth Module (12 endpoints)

3 Servers Module (10 endpoints)

4 Tickets Module (12 endpoints)

5 Credits & Store (10 endpoints)

6 Products & Categories (10 endpoints)

7 Admin Module (30+ endpoints)

8 Coin Orders (9 endpoints)

9 Email Templates (7 endpoints)

10 Invoices, Roles, AFK (15 endpoints)

11 Discord, Vouchers, Partners (15 endpoints)

12 Misc (Useful Links, Preferences, etc.)

1. AUTHENTICATION & OVERVIEW

Base URL

```
http://108.181.193.242:8080/api
```

Authentication

All protected endpoints require a JWT Bearer token in the Authorization header:

```
Authorization: Bearer <access_token>
```

Getting a Token

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "password123"
}

Response:
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": "uuid",
    "username": "user",
    "email": "user@example.com",
    "isAdmin": false,
    "coins": 100
  }
}
```

Rate Limiting

Endpoint	Limit
Global	100 requests / 60 seconds
POST /auth/register	3 requests / hour
POST /auth/login	5 requests / minute
POST /auth/2fa/*	3 requests / minute

Error Response Format

```
{
  "statusCode": 401,
  "message": "Invalid credentials",
  "error": "Unauthorized"
}
```

Auth Levels

Level	Description
PUBLIC	No authentication required
USER	Valid JWT token required

ADMIN	Valid JWT + is_admin=true required

2. AUTH MODULE

POST **/api/auth/register** PUBLIC

Register a new user account
Body: { email, username, password, first_name?, last_name? }
Response: { message, userId }

POST **/api/auth/login** PUBLIC

Login with email/username and password
Body: { email, password }
Response: { access_token, user } OR { requires2FA, email } OR { requiresEmailVerification, email }

POST **/api/auth/login-2fa** PUBLIC

Complete login with 2FA code
Body: { email, code }
Response: { access_token, user }

GET **/api/auth/me** USER

Get current user profile
Response: { id, username, email, coins, isAdmin, profilePhoto, ... }

PUT **/api/auth/me** USER

Update current user profile
Body: { username?, email?, first_name?, last_name? }

POST **/api/auth/profile/photo** USER

Upload profile photo (multipart/form-data)
Body: file (image/jpeg, image/png, max 5MB)

POST **/api/auth/setup-admin** PUBLIC

Create first admin account (requires setup key)
Body: { setupKey, username, email, password }

2FA Endpoints

POST **/api/auth/2fa/generate** USER

Generate 2FA secret and QR code
Response: { secret, qrCodeUrl }

POST **/api/auth/2fa/enable** USER

Enable 2FA with verification code
Body: { code }

POST **/api/auth/2fa/disable** USER

Disable 2FA
Body: { code }

POST **/api/auth/2fa/check** PUBLIC

Check if email has 2FA enabled
Body: { email }
Response: { requires2FA: boolean }

3. SERVERS MODULE

GET `/api/servers` USER

List all servers for current user
Response: Server[] with Pterodactyl data

POST `/api/servers` USER

Deploy a new server
Body: { serverName, productId, nestId, eggId }

GET `/api/servers/:id` USER

Get server details

PUT `/api/servers/:id` USER

Update server name
Body: { serverName }

DELETE `/api/servers/:id` USER

Delete a server

GET `/api/servers/stats` USER

Get user server statistics

GET `/api/servers/blueprint` USER

Get server deployment blueprint

GET `/api/servers/plans` USER

Get available server plans

Server Renewal

GET `/api/servers/:id/renewal-info` USER

Get server renewal cost info

POST `/api/servers/:id/renew` USER

Renew server (costs coins)

POST `/api/servers/:id/auto-renew-toggle` USER

Toggle auto-renewal

4. TICKETS MODULE

GET `/api/tickets` USER

List user's tickets

POST `/api/tickets` USER

Create a new ticket
Body: { subject, message, priority?, serverId? }

GET `/api/tickets/:id` USER

Get ticket with messages

POST `/api/tickets/:id/reply` USER

Reply to ticket
Body: { message }

POST `/api/tickets/:id/close` USER

Close ticket

Admin Tickets

GET `/api/admin/tickets` ADMIN

List all tickets

POST `/api/admin/tickets/:id/reply` ADMIN

Admin reply to ticket

POST `/api/admin/tickets/:id/close` ADMIN

Admin close ticket

5. CREDITS & STORE

GET `/api/credits/packages` USER

Get available coin packages

POST `/api/credits/buy` USER

Purchase coins (creates order)
Body: { packageId }

Coin Orders

POST `/api/coin-orders` USER

Create coin order
Body: { coins_amount, price }

GET `/api/coin-orders/list` USER

List user's coin orders

POST `/api/coin-orders/:id/screenshot` USER

Upload payment screenshot

Store (Server Upgrade/Renew)

GET `/api/store/:serverId` USER

Get upgrade options for server

POST `/api/store/:serverId/upgrade` USER

Upgrade server (RAM/CPU/Disk)
Body: { ram?, cpu?, disk? }

6. PRODUCTS & CATEGORIES

GET `/api/products` USER

List all active products

GET `/api/products/nests/:nestId/eggs` USER

Get eggs for a nest

Categories (User)

GET `/api/categories` USER

List all service categories

GET `/api/categories/:id/products` USER

Get products in a category

Admin Products

GET `/api/admin/products` ADMIN

List all products (admin)

POST `/api/admin/products` ADMIN

Create product

PUT `/api/admin/products/:id` ADMIN

Update product

DELETE `/api/admin/products/:id` ADMIN

Delete product

7. ADMIN MODULE (30+ Endpoints)

Dashboard

`GET` **/api/admin/stats** ADMIN

Get admin dashboard statistics

User Management

`GET` **/api/admin/users** ADMIN

List all users

`GET` **/api/admin/users/:id** ADMIN

Get user details

`PUT` **/api/admin/users/:id** ADMIN

Update user

`DELETE` **/api/admin/users/:id** ADMIN

Delete user

`POST` **/api/admin/users/:id/ban** ADMIN

Ban user + suspend servers

`POST` **/api/admin/users/:id/unban** ADMIN

Unban user + unsuspend servers

Server Management

`GET` **/api/admin/servers** ADMIN

List all servers

`POST` **/api/admin/servers/:id/suspend** ADMIN

Suspend server

`POST` **/api/admin/servers/:id/unsuspend** ADMIN

Unsuspend server

`POST` **/api/admin/servers/:id/delete** ADMIN

Delete server

Settings

GET `/api/admin/settings` ADMIN

Get all settings

PUT `/api/admin/settings/:key` ADMIN

Update setting
Body: { value: string }

GET `/api/admin/settings/smtp` ADMIN

Get SMTP configuration

PUT `/api/admin/settings/smtp` ADMIN

Update SMTP configuration

POST `/api/admin/email/test` ADMIN

Send test email
Body: { email: string }

Pterodactyl Integration

GET `/api/admin/pterodactyl/config` ADMIN

Get Pterodactyl config

PUT `/api/admin/pterodactyl/config` ADMIN

Update Pterodactyl config (API key, panel URL)

POST `/api/admin/pterodactyl/test` ADMIN

Test Pterodactyl connection

GET `/api/admin/pterodactyl/nodes` ADMIN

Get Pterodactyl nodes

GET `/api/admin/pterodactyl/nests` ADMIN

Get Pterodactyl nests

GET `/api/admin/pterodactyl/nests/:id/eggs` ADMIN

Get eggs for a nest

GET `/api/admin/pterodactyl/users` ADMIN

Get all Pterodactyl users

8. COIN ORDERS (Admin)

`GET` **/api/admin/coin-orders** ADMIN

List all coin orders

`GET` **/api/admin/coin-orders/pending-count** ADMIN

Get pending orders count

`POST` **/api/admin/coin-orders/:id/approve** ADMIN

Approve coin order (credits added to user)

`POST` **/api/admin/coin-orders/:id/decline** ADMIN

Decline coin order

9. EMAIL TEMPLATES

`GET` **`/api/admin/email-templates`** ADMIN

List all email templates

`POST` **`/api/admin/email-templates`** ADMIN

Create email template
Body: { name, subject, body, type, is_active? }

`PATCH` **`/api/admin/email-templates/:id`** ADMIN

Update email template

`DELETE` **`/api/admin/email-templates/:id`** ADMIN

Delete email template

`POST` **`/api/admin/email-templates/:id/test`** ADMIN

Send test email with template
Body: { email: string }
Response: { message, template }

10. INVOICES, ROLES, AFK

Invoices

GET `/api/invoices` USER

Get user's invoices

GET `/api/invoices/all` ADMIN

Get all invoices (admin)

POST `/api/invoices` ADMIN

Create invoice (admin)

Roles

GET `/api/admin/roles` ADMIN

List all roles

POST `/api/admin/roles` ADMIN

Create role

POST `/api/admin/roles/:id/assign` ADMIN

Assign role to user
Body: { userId }

AFK System

POST `/api/afk/start` USER

Start AFK earning session

POST `/api/afk/heartbeat` USER

Send heartbeat (keeps session alive)

POST `/api/afk/stop` USER

Stop AFK session + claim coins

GET `/api/afk/stats` USER

Get AFK earning stats

11. DISCORD, VOUCHERS, PARTNERS

Discord

GET `/api/discord/link` USER

Get Discord OAuth link

GET `/api/discord/callback` PUBLIC

Discord OAuth callback

GET `/api/discord/status` USER

Get Discord connection status

POST `/api/discord/unlink` USER

Unlink Discord account

Vouchers

GET `/api/admin/vouchers` ADMIN

List all vouchers

POST `/api/admin/vouchers` ADMIN

Create voucher

POST `/api/vouchers/redeem` USER

Redeem voucher code
Body: { code }

Partners

GET `/api/admin/partners` ADMIN

List all partners

POST `/api/admin/partners` ADMIN

Create partner

12. MISC ENDPOINTS

Useful Links

GET `/api/useful-links` USER

Get active useful links

GET `/api/admin/useful-links` ADMIN

List all useful links (admin)

POST `/api/admin/useful-links` ADMIN

Create useful link

Preferences

GET `/api/preferences` USER

Get user preferences

PUT `/api/preferences` USER

Update user preferences

Achievements

GET `/api/achievements` USER

Get user achievements

GET `/api/achievements/available` USER

Get available achievements

POST `/api/achievements/check` USER

Check and unlock achievements

App Config

GET `/api/config` PUBLIC

Get public app config (panel URL, etc.)

GET `/api/health` PUBLIC

Health check endpoint

Email Verification

POST `/api/auth/send-verification-email` USER

Send verification email to current user

GET `/api/auth/verify-email` PUBLIC

Verify email with token (query param: ?token=xxx)

END OF DOCUMENTATION

Zyphron Cloud API Documentation

Total Endpoints: 130+

zyphron.cloud | github.com/SirYadav1